

CI/CD Design Document - Sync Service

This document explains the CI/CD pipeline design, deployment strategy, configuration management, and rollback approach for the Spring Boot based **sync-service** application deployed on GCP virtual machines.

1. Branching Strategy

Branch Structure

The project follows a controlled Git branching model to separate development, testing, staging, and production deployments.

Branch Environment Purpose

feature/*	Developer	Local	Feature development	develop	QA Integration	and QA testing
staging	Staging	Pre-production	validation	main	Production	Stable production releases

How Branches Map To Environments

Each environment is directly connected to a dedicated branch: Merging into **develop** triggers deployment to QA. Merging into **staging** triggers deployment to Staging. Merging into **main** triggers Production deployment. This approach creates a controlled promotion flow: feature → develop → staging → main

How Accidental Production Deployments Are Prevented

Several controls are implemented: Production deployment allowed only from the **main** branch. Manual Jenkins approval before production deployment. Protected GitHub branches. Mandatory Pull Request reviews. Restricted Jenkins credentials access. No direct push access to main branch. These controls reduce deployment risk and ensure production stability.

2. Jenkins Pipeline

High-Level Pipeline Stages

The Jenkins pipeline follows these stages: **Checkout** – Clone source code from GitHub. **Build** – Build Spring Boot application using Maven. **Unit Testing** – Execute automated test cases. **SonarQube Scan** – Run static code analysis. **Docker Build** – Create Docker image. **Push Image** – Push image to private registry. **Deploy** – Deploy to target environment. **Health Check** – Validate application health.

What Happens During Pull Request

When a Pull Request is created: Code is automatically built. Unit tests are executed. SonarQube validation runs. No deployment happens. Purpose: Detect issues early. Maintain code quality. Prevent broken code merge.

What Happens During Merge

When code is merged into an environment branch: Application is built. Docker image is generated. Image is pushed to registry. Deployment is triggered automatically. Health check validation runs.

Rollback Strategy

Rollback is image-based. Each deployment stores the previous stable Docker image version. If deployment fails:

Health check failure is detected. Traffic is stopped for failed version. Previous stable image is redeployed. Application traffic is restored. This approach reduces downtime and allows fast recovery.

3. Configuration Management

Environment-Specific Configuration Management

Separate Spring Boot profile configuration files are maintained: application-qa.yml application-staging.yml application-prod.yml Each file contains: Database connection details Logging levels API endpoints Feature flags Application ports Deployment injects the required profile using: `SPRING_PROFILES_ACTIVE=qa`

Secrets Handling

Sensitive information is never stored inside GitHub repositories. Secrets are securely stored using: Jenkins Credentials Manager GCP Secret Manager Secrets include: MongoDB credentials API keys JWT secrets Third-party tokens Benefits: Improved security Centralized management Easy credential rotation Reduced leakage risk

4. Deployment Strategy

Blue/Green vs Rolling vs Recreate

Recreate Deployment

The old application version is stopped before the new version starts. This causes downtime and is not suitable for production systems.

Rolling Deployment

Application instances are updated gradually one by one. This reduces downtime but rollback can become slower and more complex.

Blue/Green Deployment

Two identical environments are maintained: Blue = Current live version Green = New version Traffic switches to Green only after successful validation.

Why Blue/Green Was Selected

Blue/Green deployment is preferred because: Near zero downtime deployment. Fast rollback capability. Reduced deployment risk. Safer production releases. Easy traffic switching.

Zero/Minimal Downtime Approach

The deployment process follows these steps: Deploy new version to Green environment. Run application health checks. Validate MongoDB connectivity. Switch traffic using Nginx reverse proxy. Keep Blue environment as backup. If issues are detected: Traffic instantly switches back to Blue. No full outage occurs. This approach ensures high availability and production stability.